The Euclidean algorithm is not optimal among uniform processes

Jason Davies

October 16, 2026

Abstract

Van den Dries and Moschovakis conjectured that the Euclidean algorithm is optimal, up to a constant factor, among the algorithms that compute the greatest common divisor from the remainder function. Their homomorphism method gives lower bounds that hold for every uniform process, deterministic or nondeterministic. It proves the binary gcd algorithm optimal, but for the Euclidean setting it has so far produced only $\Omega(\log\log a)$. We show that here the method cannot reach the conjectured $\Omega(\log a)$. Every pair $a > b \geq 1$ has a remainder-only straight-line derivation of $\gcd(a, b)$ with $O(\log a/\log\log a)$ remainder calls, where earlier values may be reused but no constants or other operations are available. The construction combines a stored Euclidean prefix with three-call contractions and two smaller recursive derivations; a refinement gives the coefficient $6 + o(1)$ when logarithms have base 2. Hence the intrinsic remainder-call complexity of the gcd and of coprimeness in the Euclidean structure is $O(\log a/\log\log a)$, and every lower bound obtained by the homomorphism method is at most this. The Euclidean algorithm is therefore not optimal among uniform processes: this refutes Conjecture 3.4 of Moschovakis's 2012 notes and the precise version (4F-5) of the Main Conjecture in his book. In the other direction, an elementary argument gives $\Omega(\log\log a)$ for almost all pairs. We also construct, for every $\varepsilon > 0$, a fixed finite nondeterministic recursive gcd program with remainder cost at most $\varepsilon \log a + B_\varepsilon$ on all positive inputs, where cost is the minimum over successful computations. The program depends on ε; no single sublogarithmic program is obtained. The Main Conjecture for algorithms, deterministic or nondeterministic, remains open, and any proof of it must use properties of algorithms beyond uniformity. The bounds, the certificate reductions, both refutations, the almost-all lower bound and the finite program family are formalized in Lean 4, including exact-cost correspondence with a specialized small-step recursive machine. The final section distinguishes the formalized results from the cited general theory.

1 Statement and relation to the book

Throughout, logarithms are to base 2. A derivation starts with its two inputs. An instruction adds $\mathrm{rem}(x, y)$, where x, y are already available and $y > 0$. Earlier values remain available. No constants are supplied. Let $G(a, b)$ be the minimum number of instructions needed to make $\gcd(a, b)$ available, and let $D(a, b) = G(a, b)$ for coprime inputs. Thus $G(a, 0) = 0$ and $D(a, 1) = 0$. These are costs of generating values.

Theorem 1. *Uniformly for integers $a > b \geq 1$,*

$$G(a, b) = O\left(\frac{\log a}{\log\log a}\right).$$

In particular this bound holds for $D(a, b)$ when $\gcd(a, b) = 1$.

As usual, the asymptotic assertion concerns sufficiently large a; a bound valid at every positive input is $O(1 + \log(2 + a)/\log\log(4 + a))$.

Background. Van den Dries and Moschovakis asked whether the Euclidean algorithm is optimal among its peers, the algorithms that compute the gcd from the remainder function [3]. Moschovakis's Main Conjecture asserts that for every algorithm α from $\mathrm{rem}, \mathrm{eq}_0$ computing $\gcd(x,y)$ there is an $r>0$ such that α makes more than $r\log x$ remainder calls on infinitely many pairs $x>y\geq 1$ [5, p. 2].

The known lower bound is a logarithm smaller. For a recursive program deciding coprimeness from $=$, $<$, $+$, truncated subtraction, iq and rem, the depth of nested calls to these primitives exceeds $\frac{1}{10}\log\log a$ on infinitely many coprime pairs [3, Theorem A]; see also [4]. The book proves it as an intrinsic bound in $\mathbf{N}_0[\div] = (\mathbb{N}, 0, 1, =, <, +, \dot{-}, \mathrm{iq}, \mathrm{rem})$: $\mathrm{depth}_{\perp\!\!\!\perp}(\mathbf{N}_0[\div], a, b) > \frac{1}{10}\log\log a$ for every "difficult pair", a class that contains every solution of Pell's equation [5, Theorem 6C.5, p. 187], so it holds for all uniform processes, nondeterministic ones included. A certificate in $\mathcal{N}_\varepsilon$ becomes one in $\mathbf{N}_0[\div]$ when each eq_0 or eq_1 fact is replaced by an equality with the constant 0 or 1, and its depth then exceeds its number of remainder facts by at most a constant. Hence $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) = \Omega(\log\log a)$ on all difficult pairs. By contrast, for the binary gcd algorithm the same method gives a matching logarithmic lower bound: a recursive program deciding coprimeness from $=$, $<$, $+$, truncated subtraction, $2x$, $\mathrm{iq}(x,2)$ and $\mathrm{rem}(x,2)$ needs more than $\frac{1}{10}\log(a^2-1)$ calls on the pair (a, a^2-1), for every $a>2$ [3, Theorem B]. Pratt's nondeterministic algorithm [5, Theorem 2E.2 and Problems x2E.17–x2E.19] matches this bound up to a constant factor on successive Fibonacci numbers [5, Corollary 6C.7]. Whether a logarithmic bound holds for all uniform processes was left open [5, p. 188], [2, Conjecture 3.4].

In a different model, Bshouty showed that a random access machine with linear operations, shifts and indirect addressing computes the gcd of two integers in $[0, N]$ in $O(\log N/\log\log N)$ operations, by reading precomputed blocks of the binary gcd algorithm from a table. He also proved that this is optimal: even with rational constants, multiplication and division by constants, and the floor function added, $\Omega(\log N/\log\log N)$ operations are necessary [1, Theorems i and ii]. The construction here uses no operation other than remainder and no table. Its saving of a factor $\log\log a$ comes instead from reusing one stored Euclidean prefix for every modulus.

The conjectures refuted. The definitions of generated certificates and intrinsic complexities are in [5, §§1D, 4C–4E] and are recalled in Section 5. Write $\mathcal{N}_0 = (\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0)$ and $\mathcal{N}_\varepsilon = (\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0, \mathrm{eq}_1)$; the structure $\mathcal{N}_0$ is not the book's $\mathbf{N}_0$, whose expansion $\mathbf{N}_0[\div]$ appears above. Write $\mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, f, \vec{x})$ for the least number of remainder facts in a certificate for f at $\vec{x}$ in the structure $\mathcal{A}$. Coprimeness is the relation $x \perp\!\!\!\perp y \iff x, y \geq 1 \wedge \gcd(x,y) = 1$ [5, §1A], [2]. Moschovakis's 2012 notes state:

> **Conjecture 3.4** [2]. There is a rational number $r>0$ such that for infinitely many $a\geq b\geq 1$, $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) \geq r\log a$.

The book formulates one precise version of the Main Conjecture [5, p. 151]: there are a set $\mathcal{D}\subseteq\mathbb{N}^2$ and a $K>0$ such that

$$(x,y)\in\mathcal{D} \implies \mathrm{calls}_{\mathrm{rem}}(\varepsilon, x, y) \leq L(x,y) \leq K\,\mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, \gcd, x, y), \tag{4F-5}$$

where $L(x,y) = 2\log\max(x,y)$ and ε is the Euclidean algorithm. The book adds that $\mathcal{D}$ then satisfies its conditions (4F-3) and (4F-4). Condition (4F-4) makes the intrinsic complexity unbounded on $\mathcal{D}$, so the intended $\mathcal{D}$ is infinite; for finite $\mathcal{D}$ the statement would be trivial. Dropping the first inequality and asking only for an infinite $\mathcal{D}$ gives a weaker statement, and we refute that weaker statement.

Corollary 2. *The intrinsic complexities*

$$\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b), \qquad \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_0, \gcd, a, b), \qquad \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \gcd, a, b)$$

are $O(\log a/\log\log a)$*, uniformly for* $a \geq b \geq 1$*. Consequently Conjecture 3.4 of [2] is false, and so is the version (4F-5) of the Main Conjecture with* $\mathcal{A} = \mathcal{N}_0$ *or* $\mathcal{A} = \mathcal{N}_\varepsilon$*: for every* $r > 0$*, respectively* $K > 0$*, only finitely many pairs satisfy the stated inequality.*

The corollary follows from Theorem 1 and Section 5. For (4F-5) it covers all pairs in $\mathbb{N}^2$: the gcd is symmetric, and a pair with a zero entry is certified by the single fact $\mathrm{eq}_0(0) = \mathrm{tt}$, with no remainder facts. It is formalized as `ShortGCD.not_conjecture_3_4`, `ShortGCD.not_main_conjecture_4F5` (for $\mathcal{N}_0$) and `ShortGCD.not_main_conjecture_4F5_Neps` (for $\mathcal{N}_\varepsilon$). The literal readings, with a rational r and infinitely many a for Conjecture 3.4, and with a set $\mathcal{D}$ on which the complexity is unbounded for (4F-5), are `ShortGCD.not_conjecture_3_4_literal` and `ShortGCD.not_main_conjecture_4F5_literal`.

Theorem 4F.1 [5, pp. 149–150] supplies a single uniform process that attains these pointwise intrinsic bounds. So some uniform process of $\mathcal{N}_\varepsilon$ decides coprimeness, and one computes the gcd, with $O(\log a/\log\log a)$ remainder calls on all pairs $a \geq b \geq 1$. This settles the possibility raised after Corollary 6C.7 [5, p. 188]: the Main Conjecture does not hold for all uniform processes of $\mathcal{N}_\varepsilon$.

The limit of the homomorphism method. The homomorphism method proves lower bounds on intrinsic complexities. These bound every uniform process that computes the function: if α computes f in $\mathcal{A}$, then $\mu(\mathcal{A}, f, \vec{x}) \leq \mu(\alpha, \vec{x})$, in particular for every deterministic or nondeterministic recursive program [5, Proposition 4E.1, Corollary 4E.2]. The Homomorphism Test [5, Lemma 4E.3] proves a lower bound on $\mu(\mathcal{A}, f, \vec{x})$ itself, by showing that every generated diagram below the proposed cost admits a homomorphism that fails to preserve the answer. Corollary 2 caps every such argument.

Corollary 3 (Limit of the homomorphism method)**.** *Let* $g(a, b)$ *be any lower bound for* $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b)$*, or for* $\mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, \gcd, a, b)$ *with* $\mathcal{A} = \mathcal{N}_0$ *or* $\mathcal{N}_\varepsilon$*, for instance one obtained from the Homomorphism Test. Then* $g(a, b) = O(\log a/\log\log a)$*, uniformly for* $a \geq b \geq 1$*. In particular, no lower bound on intrinsic complexity shows that every algorithm from* $\mathrm{rem}, \mathrm{eq}_0$ *computing the gcd makes at least* $r \log a$ *remainder calls on infinitely many pairs. A proof of the Main Conjecture must use properties of algorithms that uniform processes need not have, such as being induced by a finite program, or determinism.*

Proof. Immediate from Corollary 2. □

The corollary does not exclude arguments that combine homomorphisms with additional restrictions on computations.

Corollaries 2 and 3 do not settle the Main Conjecture for algorithms, that is, recursive programs, deterministic or nondeterministic [5, p. 2]. Nor do they settle the form for deterministic uniform processes [2, Conjecture 3.9], [5, §4H and p. 188]; see Section 8. With the lower bound above, the intrinsic complexity of coprimeness in $\mathcal{N}_\varepsilon$ now lies between $\Omega(\log\log a)$, on infinitely many pairs and in fact on almost all pairs (Proposition 8), and $O(\log a/\log\log a)$, on all pairs. Closing this gap is an open problem. In particular, it is not known whether the bound of Theorem 1 is optimal for remainder-only derivations, as the corresponding bound is in Bshouty's model.

There is also an algorithmic consequence with a different quantifier order. Section 6 constructs fixed finite nondeterministic recursive programs with arbitrarily small logarithmic remainder-call coefficients, uniformly over all inputs. The programs use a fixed-length prefix and ordinary recursive

calls. This complements Pratt's bound on successive Fibonacci numbers: the saving here is uniform, while the coefficient is chosen by enlarging the program. The Main Conjecture allows its lower-bound coefficient to depend on the program, so this consequence is compatible with it.

2 Normalization

Write $g = \gcd(a, b) > 0$. For positive y,

$$\operatorname{rem}(gx, gy) = g \operatorname{rem}(x, y).$$

Every generated value is divisible by g. Consequently scaling a derivation, or dividing all its values by g, preserves its length and the validity of every instruction. Hence

$$G(a, b) = D(a/g, b/g). \tag{1}$$

Define

$$T(n) = \max_{0 \le b \le a \le 2^n,\ a>0} G(a, b).$$

Euclid's algorithm makes this a finite maximum. Normalization reduces an upper bound for T to coprime inputs, with equality, zero inputs, and inputs equal to one handled without calls.

3 Three-call contractions

Lemma 4. *If $m > 0$ is available and a finite collection S of available values occupies $t \ge 2$ residue classes modulo m, at most three remainder calls generate w with*

$$0 < w < \frac{m}{t-1}.$$

Proof. Sort the distinct residues. Some adjacent pair $u < v$ has gap $\delta = v - u < m/(t-1)$, because the sum of the $t-1$ adjacent gaps is less than m. Each selected residue can be generated with one call from a witness in S. If $u = 0$, use v. If $0 < u \le \delta$, use u. Otherwise $0 < \delta < u$ and $v = u + \delta$, so $\operatorname{rem}(v, u) = \delta$. This uses at most three calls. Subtraction and sorting describe the proof and the selection of instructions; they are not instructions of the derivation. □

Lemma 5 (Euclidean-prefix lemma)**.** *Let $r_0 = a > r_1 = b > \cdots > r_h > 0$ be a Euclidean prefix of a coprime pair, with $r_{i-1} = q_i r_i + r_{i+1}$ and $q_i \ge 1$ for $1 \le i < h$. For every integer $m > 2a/r_h$, the h pairs*

$$(r_i \bmod m, r_{i+1} \bmod m), \qquad 0 \le i < h,$$

are distinct. The prefix therefore occupies t residue classes with $t^2 \ge h$.

Proof. Suppose the pairs at $i < j$ coincide. Set

$$M = \prod_{\ell=i+1}^{j} \begin{pmatrix} q_\ell & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} A & B \\ C & E \end{pmatrix}, \qquad \begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix} = M \begin{pmatrix} r_j \\ r_{j+1} \end{pmatrix}.$$

Right multiplication by $\left(\begin{smallmatrix} q & 1 \\ 1 & 0 \end{smallmatrix}\right)$ sends (A, B, C, E) to $(qA+B,\ A,\ qC+E,\ C)$, starting from $(q, 1, 1, 0)$ for a single factor. By induction, all entries are nonnegative, $A \ge B \ge E$, $A \ge C \ge 1$, $B \ge 1$, and $\det M = (-1)^{j-i}$. Also $Ar_j \le r_i \le a$, whence $A \le a/r_h$. Write

$$\Delta = \det(M - I) = 1 + \det M - A - E.$$

For an odd number of factors, $\Delta = -A - E < 0$. By the same recursion, a product of at least two factors has $A \geq 2$ and $E \geq 1$; thus for an even positive number of factors, $\Delta = 2 - A - E < 0$. In either case,

$$0 < |\Delta| \leq A + E \leq 2A \leq 2a/r_h < m.$$

The presumed congruence gives $(M - I)(r_j, r_{j+1})^{\mathsf{T}} \equiv 0 \pmod{m}$. Multiplication by the adjugate shows that m divides both Δr_j and Δr_{j+1}. Consecutive remainders are coprime, so Bézout's identity implies $m \mid \Delta$, a contradiction. Finally, t individual residue classes give at most t^2 ordered pairs. □

Thus a stored prefix can be reused for every available modulus $m > 2a/r_h$. When $\sqrt{h} > 1$, three calls suffice to generate

$$0 < w < \frac{m}{\sqrt{h} - 1}. \tag{2}$$

The universal quantifier over m is essential: the modulus generated by a contraction is not predetermined.

4 Construction and recurrence

Fix an integer $n > 16$ and put $N = 2^n$ and $h = \lceil \sqrt{n} \rceil$, so that $h > 4$. After normalization we may assume $a > b > 1$ and $a \leq N$. Generate the Euclidean prefix until its last positive value is at most $N^{3/4}$ or until r_h is available. Check the input b as part of this rule. The preliminary work costs at most h calls. Euclid cannot encounter zero first without having encountered the positive remainder 1.

If a positive remainder $c \leq N^{3/4}$ is encountered, generate the next remainder $s < c$ with one more call. The pair (c, s) is coprime. If $s = 0$, then $c = 1$ is already available; otherwise both inputs to the remaining derivation are at most $2^{\lceil 3n/4 \rceil}$. This gives the bound

$$h + 1 + T(\lceil 3n/4 \rceil). \tag{3}$$

Otherwise $r_h > N^{3/4}$, so $2a/r_h < 2N^{1/4}$. For $n > 12$, $N^{1/3} > 2N^{1/4}$, and Lemma 5 applies to every available $m > N^{1/3}$. Starting at $m = r_h$, repeatedly use the contraction inequality until $0 < m \leq N^{1/3}$. Set

$$\kappa_n = \sqrt{h} - 1 > 1, \qquad J_n = \left\lceil \frac{2n/3}{\log \kappa_n} \right\rceil.$$

There are at most J_n contractions: if J_n have occurred then $m < N/\kappa_n^{J_n} \leq N^{1/3}$. Since $\log \kappa_n = \frac{1}{4} \log n + O(1)$, their cost is $3J_n = O(n/\log n)$.

The small m need not be coprime to either input. Generate $u = \mathrm{rem}(a, m)$ and $v = \mathrm{rem}(b, m)$, then concatenate derivations of $d = \gcd(m, u)$ and $e = \gcd(d, v)$. These derivations exist by the definition of T, since G is symmetric in its arguments, and use only inputs bounded by m. They are not gcd-oracle instructions. The gcd identities give

$$e = \gcd(\gcd(m, a \bmod m), b \bmod m) = \gcd(m, \gcd(a, b)) = 1.$$

The cost in this case is at most

$$h + 3J_n + 2 + 2T(\lceil n/3 \rceil). \tag{4}$$

Retaining values makes concatenation legitimate. In particular, the second recursive derivation has both d and v available even if the first one uses neither original input.

The two case estimates and normalization imply

$$T(n) \le \max\{T(\lceil 3n/4 \rceil) + O(\sqrt{n}),\ 2T(\lceil n/3 \rceil) + O(n/\log n)\}. \tag{5}$$

All implied constants are independent of the inputs.

Proof of Theorem 1. For $n \ge 2$ let $f(n) = n/\log n$. The ratios

$$\frac{f(\lceil 3n/4 \rceil)}{f(n)} \longrightarrow \frac{3}{4}, \qquad \frac{2f(\lceil n/3 \rceil)}{f(n)} \longrightarrow \frac{2}{3}$$

show that, beyond a fixed threshold, $f(\lceil 3n/4 \rceil) \le \frac{7}{8} f(n)$ and $2f(\lceil n/3 \rceil) \le \frac{5}{6} f(n)$. Both overheads in (5) are at most $c_0 f(n)$ for a fixed c_0. Choose the induction threshold above 16, where the recurrence holds, and large enough that both recursive indices are at least 2 and strictly less than n. Choose $C \ge 8c_0$, increased to cover the finitely many values below the threshold. Strong induction then gives $T(n) \le Cf(n)$: the two bounds are $(7C/8 + c_0)f(n)$ and $(5C/6 + c_0)f(n)$, both at most $Cf(n)$. Finally take $n = \lceil \log a \rceil$. This also gives $G(a, b) = o(\log a)$ uniformly as $a \to \infty$. □

4.1 The leading coefficient

The same construction allows a more precise bound without changing either arithmetic lemma.

Corollary 6. *For every $\varepsilon > 0$, there is an a_0 such that for every $a \ge a_0$ and $0 \le b \le a$,*

$$G(a, b) \le (6 + \varepsilon) \frac{\log a}{\log \log a}.$$

Equivalently, the uniform upper bound has coefficient $6 + o(1)$.

Proof. Fix $0 < c < 1/2$ and $0 < \beta < 1/2$, and put $\alpha = 1 - \beta/2$. For sufficiently large n, choose

$$p = \lfloor c \log n \rfloor, \qquad k = 2^p, \qquad h = k^2 + 1, \qquad J = \lceil n/p \rceil.$$

Use N^α as the early stopping threshold and N^β as the small-modulus threshold, where $N = 2^n$. Since $\alpha + \beta > 1$, the prefix lemma applies to every modulus above N^β for sufficiently large n. Since $k^2 < h$, the prefix occupies at least $k + 1$ residue classes, so each contraction reduces the modulus by a factor greater than k. Moreover $k^J \ge N$, so J contractions suffice. Writing $f(n) = n/\log n$, we have

$$h = O(n^{2c}) = o(f(n)), \qquad 3J = \left(\frac{3}{c} + o(1) \right) f(n).$$

The two branches therefore give

$$T(n) \le \max \left\{ T(\lceil \alpha n \rceil) + o(f(n)), \quad 2T(\lceil \beta n \rceil) + \left(\frac{3}{c} + o(1) \right) f(n) \right\}.$$

Theorem 1 implies that $L = \limsup_{n \to \infty} T(n)/f(n)$ is finite and nonnegative. For each fixed $q > 0$, the ratio $f(\lceil qn \rceil)/f(n)$ tends to q. Taking upper limits in the recurrence yields

$$L \le \max\{\alpha L,\ 2\beta L + 3/c\}.$$

As $\alpha < 1$, this implies $L(1 - 2\beta) \le 3/c$; the case $L = 0$ is immediate. First let $\beta \downarrow 0$ and then $c \uparrow 1/2$ to obtain $L \le 6$. Finally, $n = \lceil \log a \rceil$ satisfies $f(n) \sim \log a / \log \log a$, proving the assertion. All upper limits concern the worst-case function T, so the resulting threshold is independent of b. □

No optimality of the coefficient 6 is asserted. The refinement is formalized as `ShortGCD.uniform_real_bound_six`; the original fixed parameters suffice for the simpler Big-O proof above.

5 Certificates and uniform processes

Use the book's partial remainder operation: its second argument must be nonzero. Recall $\mathcal{N}_0 = (\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0)$ and $\mathcal{N}_\varepsilon = (\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0, \mathrm{eq}_1)$; compare [5, (1C-10), pp. 24, 142–143]. A derivation determines a finite generated partial substructure by recording its remainder equations. Repeated equations can be discarded.

Consider first the relation $\gcd(a, b) = 1$ on all natural inputs. A derivation of 1 together with $\mathrm{eq}_1(1) = \mathrm{tt}$ certifies the relation: every common divisor of the images of the inputs divides every transported generated value, hence divides 1. Conversely, without a positive eq_1 fact the map $x \mapsto 2x$ preserves every recorded remainder and zero-test fact, as well as every negative one-test fact. It cannot preserve coprimeness. A certificate therefore reaches 1. Enumerating a finite generated diagram in generation order gives a derivation using no more calls than the number of remainder facts. Consequently, on coprime inputs,

$$\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \gcd{=}1, a, b) = D(a, b), \qquad \mathrm{calls}(\mathcal{N}_\varepsilon, \gcd{=}1, a, b) = D(a, b) + 1.$$

For the book's relation $\perp\!\!\!\perp$, which also requires $a, b \geq 1$, it suffices to add the facts $\mathrm{eq}_0(a) = \mathrm{ff}$ and $\mathrm{eq}_0(b) = \mathrm{ff}$. Conversely, when $a \perp\!\!\!\perp b$, every certificate for $\perp\!\!\!\perp$ also certifies $\gcd(a, b) = 1$. So for coprime $a, b \geq 1$ the remainder count is unchanged, $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) = D(a, b)$, and the total count is at most $D(a, b) + 3$. For noncoprime inputs D plays no role; the bound there comes from the gcd certificate below.

For gcd as a function, the certificate must reach its output g, giving a lower bound of $G(a, b)$. For an upper bound, append

$$\mathrm{rem}(a, g) = z, \qquad \mathrm{rem}(b, g) = z, \qquad \mathrm{eq}_0(z) = \mathrm{tt},$$

where the generated z is zero. Every homomorphism π has $\pi(z) = 0$ and $\pi(g) > 0$ (by definedness of either appended remainder). Thus $\pi(g)$ divides both input images. Conversely every common divisor of those images divides $\pi(g)$ by induction over the derivation. Therefore, for $(a, b) \neq (0, 0)$,

$$G(a, b) \leq \mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_0, \gcd, a, b) \leq G(a, b) + 2.$$

At $(0, 0)$ the output is an input and the empty certificate suffices. Positivity facts may be appended if the function is restricted to positive inputs. For a noncoprime input, add $\mathrm{eq}_1(g) = \mathrm{ff}$ to this gcd certificate to certify noncoprimeness. Thus the sublogarithmic remainder-call bound holds for both truth values of coprimeness as well as for gcd. The fixed certificate overhead does not change the leading coefficient. Thus Corollary 6 also holds for each of the three intrinsic complexities in Corollary 2, on positive inputs. This is `ShortGCD.uniform_intrinsic_bound_six`. Theorem 4F.1 supplies a single uniform process realizing each intrinsic norm pointwise [5, Theorem 4F.1].

6 Fixed programs with arbitrarily small coefficients

For a nondeterministic program P, let $c_P(a, b)$ be the minimum number of remainder calls among its convergent computations. We require at least one such computation on each input and require every convergent computation to return the gcd. These are the conventions of [5, (2E-2), (2E-5)].

Theorem 7. *For each integer $p \geq 1$, there is a finite nondeterministic recursive program P_p over $(\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0)$ computing the gcd such that*

$$\limsup_{a \to \infty} \max_{1 \leq b \leq a} \frac{c_{P_p}(a, b)}{\log a} \leq \frac{3}{p}.$$

Its generator has $W = 4^p + 5$ individual parameters, and no internal function has larger arity. Consequently, for every $\varepsilon > 0$ there are a fixed program P and a constant $B \geq 0$ such that

$$\forall a \geq b \geq 1, \qquad c_P(a, b) \leq \varepsilon \log a + B.$$

The program. Fix $W \geq 3$ and write $\vec{x} = (x_0, \ldots, x_{W-1})$ and $\vec{x}[i \leftarrow E]$ for the tuple with its ith expression replaced by E. The generator is given by the finite alternatives

$$\begin{aligned}
\mathrm{choose}(\vec{x}) &= x_i && (0 \leq i < W),\\
\mathrm{gen}(\vec{x}) &= \mathrm{choose}(\vec{x}),\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\vec{x}[i \leftarrow \mathrm{rem}(\mathrm{choose}(\vec{x}), \mathrm{choose}(\vec{x}))]) && (0 \leq i < W),\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\vec{x}[i \leftarrow \mathrm{gen}(\vec{x})]) && (0 \leq i < W),\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\mathrm{choose}(\vec{x}), \ldots, \mathrm{choose}(\vec{x})).
\end{aligned}$$

There are W independent selector calls in the last line. Tuple notation and indices are syntax abbreviations: after fixing W, each line expands into ordinary first-order terms with fixed arities. No tuple, index, numeric constant, or update operation is supplied as runtime data or a primitive. The nested generator in the fourth line is evaluated once; its caller's other parameters remain available when it returns. The recursion stack can grow, so W is an arity bound, not a total storage bound.

Add $\mathrm{loop}(z) = \mathrm{loop}(z)$ and define $\mathrm{accept}(a, b, g)$ by nested zero-test conditionals: if $g = 0$, loop; otherwise test $\mathrm{rem}(a, g) = 0$ and then $\mathrm{rem}(b, g) = 0$, returning g if both tests succeed and looping otherwise. The head is

$$P_W(a, b) = \text{if } \mathrm{eq}_0(b) \text{ then } a \text{ else } \mathrm{accept}(a, b, \mathrm{gen}(a, b, a, \ldots, a)).$$

The four internal symbols have arities $W, W, 3, 1$. Every common divisor of the generator's parameters divides its output, by induction on a finite evaluation. Thus a candidate is divisible by $\gcd(a, b)$; successful verification proves the reverse divisibility. Every convergent computation therefore returns the gcd. There is always a convergent computation following Euclid, using parameter selection and copying to retain the current pair. For $b > 0$, final verification costs exactly two remainder calls.

Operational correspondence. The syntax above is a finite recursive program in the sense of [5, §§2A, 2E]. In Lean we specialize the book's two-stack transition table to this language and program family. Closures represent substitution of actual values into terms; they are machine states, not additional program data. The control stack has its top at the head, and the value stack distinguishes individuals from booleans. Argument evaluation, zero testing, and branching are explicit transitions. The finite evaluation trees correspond to convergent machine computations, preserving both result and remainder count. For the forward direction, induct on the evaluation tree. Evaluate argument subtrees right to left, as prescribed by the machine's composition rule, then choose the indicated body at the internal call. Transition locality permits each subtree computation to be placed in its calling context [5, Table 1 and Lemma 2D.1, pp. 75–76]. A remainder node adds one external remainder call; a conditional evaluates its guard, makes one uncharged zero test, and evaluates only the selected branch. Variables and internal calls add no remainder calls. For the reverse direction, decompose a convergent computation at the completed argument evaluations and the selected body or branch, and induct on its length. This gives the same additive cost equations. Choices belong to individual call occurrences, and the language has no shared mutable

state, so serializing the argument trees introduces no constraint on their choices. No successful tree uses a zero divisor or terminates through loop. The exact-cost equivalence is checked as `ShortGCD.RegisterProgram.Syntax.Machine.eval_iff_steps`; transition locality is checked as `Machine.Step.context` in the same namespace. This formalizes the machine needed here, not the book's theory for arbitrary structures and program vocabularies.

Proof of Theorem 7. Set $k = 2^p$, $h = k^2 + 1$, and $W = h + 4$, and take $P_p = P_W$. Store $r_0, \ldots, r_h$ in $h + 1$ parameters, reserving three parameters for the modulus and two temporary residues. Lemmas 4 and 5 give a contraction by a factor greater than k in at most three remainder calls, preserving the prefix. The program can choose precisely the operands that witness each contraction.

Let $T_W(n)$ be the maximum, over $a, b \le 2^n$, of the minimum generator cost to return $\gcd(a, b)$ from $(a, b, a, \ldots, a)$. Euclid gives $T_W(n) = O(n)$. Copying parameters gives symmetry, and positive scaling preserves generator runs and their remainder counts. It therefore suffices to analyze coprime inputs.

Fix $0 < \beta < 1/2$ and put $\alpha = 1 - \beta/2$. Use the early threshold N^α and the target modulus N^β, where $N = 2^n$. If the prefix is large, $2a/r_h < 2N^{1-\alpha} < N^\beta$ for all sufficiently large n. There are at least $k + 1$ distinct prefix residues modulo every modulus used before stopping, since $h > k^2$. At most $\lceil n/p \rceil$ contractions produce a positive $m \le N^\beta$.

Form $u = a \bmod m$ and $v = b \bmod m$ in the temporary positions. A nested generator call, with its parameters routed to copies of m, u, returns $d = \gcd(m, u)$ while its caller retains v. Replace the modulus parameter by d and route to copies of d, v for the second gcd computation. The result is $\gcd(m, a, b) = 1$. For general inputs the scaled witness returns the original gcd. In the early case, route directly to the smaller consecutive Euclidean pair. All these routes and nested calls are available in the same fixed program. Thus, eventually in n,

$$T_W(n) \le \max\left\{T_W(\lceil \alpha n \rceil) + h + 1, \quad 2T_W(\lceil \beta n \rceil) + h + 3\lceil n/p \rceil + 2\right\}.$$

Set $L_W = \limsup_{n\to\infty} T_W(n)/n$, finite and nonnegative by Euclid. Since h is fixed, division by n and passage to upper limits gives

$$L_W \le \max\{\alpha L_W,\ 2\beta L_W + 3/p\}.$$

As $\alpha < 1$, this implies $(1 - 2\beta)L_W \le 3/p$. The program is independent of β, so letting $\beta \downarrow 0$ gives $L_W \le 3/p$. Take $n = \lceil \log a \rceil$ and add the two verification calls. Finally, choose $p > 3/\varepsilon$ and absorb the finitely many smaller inputs in B. □

The theorem is a statement about a family of programs. The prefix length grows when a smaller coefficient is requested, but remains fixed as the inputs grow. It does not exchange $\forall\varepsilon\,\exists P_\varepsilon$ with $\exists P\,\forall\varepsilon$. Neither the coefficient nor the arity bound is claimed optimal. Merely inlining several remainder instructions into one term would still charge for each external remainder call; the saving here uses the prefix lemma and recursive reduction.

7 An almost-all lower bound

The difficult pairs of [5, Theorem 6C.5] give $\Omega(\log\log a)$ on a sparse set. An elementary Diophantine argument gives the same order for almost all pairs, simultaneously for every derivation.

Proposition 8. *For every $d \ge 1$ and $N \ge 3$, at most $16(2 + \ln N)N^{2-2^{-d}}$ pairs $2 \le b < a \le N$ satisfy $D(a, b) \le d$. Hence $D(a, b) \ge (1 - o(1))\log\log N$ for all but $o(N^2)$ coprime pairs below N.*

Proof. For $B \geq 1$ call (a, b) *B-regular* if $|qb - pa| \geq a/(Bq)$ whenever $q \geq 1$, $p \in \mathbb{Z}$ and $qb \neq pa$.

- **Small values have large coefficients.** Every generated value is an integer form $v = ua+wb$. If (a, b) is B-regular and $v > 0$, then $v \geq a/(B(|u| + |w|))$: for $w \neq 0$ apply regularity with $q = |w|$, and for $w = 0$ note $v \geq a$.
- **Coefficients grow at most doubly exponentially.** Let H_t bound $|u| + |w|$ over the values available after t steps, so $H_0 = 1$. A step $x = qy + r$ has $q \leq a/y \leq BH_t$, hence $H_{t+1} \leq H_t + BH_t^2 \leq 2BH_t^2$ and $H_t \leq (2B)^{2^t-1}$.
- **Regular pairs need long derivations.** Producing 1 within d steps forces $1 \geq a/(BH_d)$, that is $a \leq (2B)^{2^d}/2$.
- **Few pairs are irregular.** Suppose $0 < |qb - pa| < a/(Bq)$. Dividing p, q by their gcd preserves this, and it forces $q < a/B \leq N/B$ and $0 \leq p \leq q$. For each q there are at most $2q$ values of p and $2N/(Bq)$ values of $r = qb - pa$. Given p, q, r with $\gcd(p, q) = 1$, a lies in one residue class modulo q, which leaves at most $N/q + 1$ choices, and b is then determined. So at most $(4N^2/B)(2 + \ln N)$ pairs below N are irregular.
- **Dyadic ranges.** For $T/2 < a \leq T$ take $B = T^{2^{-d}}/2$. If $B \geq 1$, a regular pair with $D \leq d$ would have $a \leq T/2$, so every pair in the range with $D \leq d$ is irregular. That gives at most $8(2 + \ln T)T^{2-2^{-d}}$ pairs, and the same bound is trivial when $B < 1$. Summing over $T = N, N/2, \ldots$ gives the first claim.

For the second claim take $d = \lfloor \log_2 \ln N - 2 \log_2 \ln \ln N \rfloor$. Then $N^{-2^{-d}} \leq \exp(-(\ln \ln N)^2)$, so the bound is $o(N^2)$. □

Lean checks Proposition 8 with the constant $40(2+\ln N)$ in place of $16(2+\ln N)$: it takes $B = a^{2^{-d}}/2$ separately for each pair instead of dyadic ranges (`ShortGCD.AlmostAll.card_within_le`, `ShortGCD.AlmostAll.card_D_le`), and it proves the second claim in the form $D(a, b) > (1 - \varepsilon) \log_2 \log_2 N$ for all but $o(N^2)$ pairs, for every $\varepsilon \in (0, 1]$ (`ShortGCD.AlmostAll.tendsto_card_within_loglog`). The saving 2^{-d} shrinks as d grows, so this argument cannot reach the scale $\log N / \log \log N$ of Theorem 1, let alone $\log N$.

8 Scope and verification

Selecting a close pair of residues is uncharged existential choice in a straight-line derivation. A deterministic remainder-only algorithm cannot assume that choice is available at constant cost. Computing all prefix residues would take about h calls per contraction. Nondeterministic operand selection and the fixed-prefix programs of Section 6 leave a further question: can one fixed first-order recursive program retain and repeatedly access the growing prefix within the same remainder-call bound? Proposition 2E.1 converts each certificate into a computation of some program; it does not supply one program for all certificates [5, §2E]. The book also distinguishes the best uniform process from a nondeterministic program [5, p. 150]. The construction therefore settles neither algorithmic version of the Main Conjecture.

The supplied offline Python planner uses the original construction parameters and unrestricted arithmetic to select instructions; its independent replay checks operand availability, nonzero divisors, every remainder equation, and the final gcd. Below a base size (256 bits by default) it uses Euclid's algorithm. On one pseudorandom coprime pair of each size, generated from its default seed, it gives:

Input bits	Derivation calls	Euclid calls to reach 1, same pair
512	196	326
2048	669	1204
8192	2059	4754

Random pairs are not Euclid's worst case: on successive Fibonacci numbers Euclid needs about $1.44 \log a$ calls. These are checks of finite instances, not an asymptotic proof. A reproducible suite of 310 cases also passes, including 175 noncoprime pairs, with base size 64 bits and 3,543 contractions in total, and four deliberately invalid traces are rejected. The Euclidean-prefix lemma was also checked exhaustively for every coprime pair with $3 \le a < 1500$ and $1 \le b < a$, every prefix length, and up to 60 consecutive moduli above the threshold $2a/r_h$, restricted to $m < 4a$ (254,722,179 instances, no failures).

8.1 Machine-checked statements

The Lean 4.35.0 development, with mathlib pinned to version 4.35.0, proves the uniform real-logarithmic bound, its worst-case Big-O formulation, the leading coefficient in Corollary 6, the Euclidean-prefix lemma, the complete three-call contraction lemma, normalization, and the certificate upper bounds. It contains no proof placeholders or added axioms. An axiom audit of the main results reports only the standard Lean foundations `propext`, `Classical.choice`, and `Quot.sound`. The theorem `ShortGCD.uniform_real_bound` states

$$\exists C > 0 \; \forall a, b \in \mathbb{N}, \quad 4 \le a \;\wedge\; b \le a \implies G(a,b) \le C \frac{\log_2 a}{\log_2(\log_2 a)}.$$

The sharper theorem `ShortGCD.uniform_real_bound_six` states that for every $\varepsilon > 0$, eventually in a, the same inequality holds for all $b \le a$ with $C = 6+\varepsilon$. `ShortGCD.uniform_intrinsic_bound_six` gives the corresponding statement for the three intrinsic complexities on positive inputs. The proof starts from a finite-store instruction semantics in which the only instruction is remainder with a positive, available divisor. Costs are indices of derivations, rather than an assumed recurrence. A formal conversion to finite generated diagrams proves the gcd certificate bound with two extra remainder facts and the bound for positive and negative coprimeness instances. The reduction itself is also checked: on coprime inputs, the least number of remainder facts in a certificate for $\gcd(a,b) = 1$ equals $D(a,b)$ (`ShortGCD.callsRem_eq_D`), and G agrees with D (`ShortGCD.G_eq_D`).

The Lean definitions from certificates are `ShortGCD.callsCoprimeness` for the book's relation $\perp\!\!\!\perp$ in $\mathcal{N}_\varepsilon$, with positive inputs (`ShortGCD.BookCoprime`), and `ShortGCD.callsGcd` and `ShortGCD.callsRemGcd` for the gcd in $\mathcal{N}_0$ and $\mathcal{N}_\varepsilon$. Adding the facts $\mathrm{eq}_0(a) = \mathrm{ff}$ and $\mathrm{eq}_0(b) = \mathrm{ff}$ is checked to turn a certificate for $\gcd(a,b) = 1$ into one for $\perp\!\!\!\perp$ (`ShortGCD.certifiesCoprimeness_of_certifiesCoprime`), so on positive coprime pairs the complexity agrees with the reduction (`ShortGCD.callsCoprimeness_eq_callsRem`).

The theorems `ShortGCD.not_conjecture_3_4`, `ShortGCD.not_main_conjecture_4F5` and `ShortGCD.not_main_conjecture_4F5_Neps` state that no $r > 0$ satisfies $\mathrm{calls}_{\mathrm{rem}}(\mathcal{N}_\varepsilon, \perp\!\!\!\perp, a, b) \ge r \log_2 a$ for infinitely many $a \ge b \ge 1$, and that no $K > 0$ satisfies $2 \log_2 \max(x,y) \le K \, \mathrm{calls}_{\mathrm{rem}}(\mathcal{A}, \gcd, x, y)$ for infinitely many pairs in $\mathbb{N}^2$, for either structure. The original $O(\log a / \log\log a)$ bounds combine `ShortGCD.callsCoprimeness_le` and `ShortGCD.callsGcd_le`, which bound the intrinsic complexities by $G(a,b) + 2$, with `ShortGCD.uniform_real_bound`; `ShortGCD.eventually_calls_le` states the resulting $o(\log)$ bound. The general theory of uniform processes, Theorem 4F.1 and the lower bounds of Section 1 are cited, not formalized. Theorem 4F.1

is needed only to restate these intrinsic bounds in terms of a single best uniform process, not for the refutations themselves.

Theorem 7 is checked in the eight modules `RegisterProgram` through `RegisterSmallStep`. Under `ShortGCD.RegisterProgram`, the theorem `maxCost_step` proves the fixed-parameter recurrence, and `Fixed.limsup_profile_le` proves its coefficient $3/p$. The syntax development gives finite body sets for the four internal symbols above. `Syntax.run_invokes` translates generator runs into evaluation trees with exactly the same remainder count; `Syntax.accepts_eval` includes final verification. `Syntax.eval_head_correct` proves correctness of every terminating evaluation, and `Syntax.eval_head_exists` proves existence on every input. The quantified bound is `Syntax.arbitrarily_small_program`; `Syntax.eventually_eval_bound` gives every coefficient greater than $3/p$ for a fixed p. `Syntax.Machine.eval_iff_steps` proves the correspondence with the specialized small-step machine in both directions. `Syntax.Machine.head_correct` and `Syntax.Machine.head_exists` give correctness and convergence, while `Syntax.Machine.arbitrarily_small_program` states the bound directly for finite transition sequences. These modules are included in the normal build and axiom audit. An independent Python compiler and right-to-left interpreter check 138 cases across three fixed widths, 129 additional scaled replays, and eight negative cases. The interpreter is not formally verified; unrestricted arithmetic used to plan successful choices is not part of the programs.

Proposition 8 is checked in the module `ShortGCD.AlmostAll`. `ShortGCD.AlmostAll.card_within_le` bounds the number of pairs $2 \le b < a \le N$ with a derivation of 1 of length at most d by $40(2+\ln N)N^{2-2^{-d}}$; `ShortGCD.AlmostAll.within_iff_D_le` identifies this with $D(a,b) \le d$ on coprime pairs; and `ShortGCD.AlmostAll.tendsto_card_within_loglog` gives the almost-all bound $D > (1-\varepsilon)\log_2\log_2 N$.

8.2 Integer parameters used in Lean

To avoid real rounding in the construction itself, the formal proof uses, for $j = \lfloor \log_2 n \rfloor \ge 32$,

$$p = \lfloor j/4 \rfloor, \quad k = 2^p, \quad h = 2^{2p}+1, \quad J = \lfloor n/p \rfloor + 1, \quad \ell_1 = \lceil 3n/4 \rceil, \quad \ell_2 = \lceil n/3 \rceil.$$

Write $\widehat{T}(X) = \max_{a,b \le X} G(a,b)$. The finite construction proves, for arbitrary natural parameters satisfying $k \ge 1$, $k^2 < h$, $Y \ge 1$, $2X \le ZY$, and $X \le Zk^J$,

$$\widehat{T}(X) \le \max\{h+1+\widehat{T}(Y),\ h+3J+2+2\widehat{T}(Z)\}.$$

Taking $X = 2^n$, $Y = 2^{\ell_1}$, $Z = 2^{\ell_2}$ gives the required hypotheses. Lean checks

$$(h+1)j \le 64n, \quad (h+3J+2)j \le 64n, \quad j-2 \le \log_2^{\mathrm{nat}} \ell_1, \log_2^{\mathrm{nat}} \ell_2, \quad 8\ell_1 \le 7n, \quad 16\ell_2 \le 7n.$$

The last estimates and $8j \le 9(j-2)$ give a factor $63/64$ for either recursive contribution after clearing denominators. Taking $C \ge 4096$ and large enough for the finite initial segment proves $\widehat{T}(2^n)\lfloor \log_2 n \rfloor \le Cn$ for all n. Comparing integer floors and ceilings with real base-two logarithms yields the stated real bound. These parameter choices change only absolute constants, not the proof mechanism or its asymptotic order. The refinement in `ShortGCD.SharpBound` uses the tunable parameters of Corollary 6, with recursive size bounds $2^{\lceil \alpha n \rceil}$ and $2^{\lceil \beta n \rceil}$. It applies the same finite recurrence, uses the original bound to justify a finite upper limit, and checks the rounding limits needed to preserve the coefficient 6.

References

[1] Nader H. Bshouty. *Euclidean GCD algorithm is not optimal*. Research Report 89/376/38, Department of Computer Science, University of Calgary, 1989. doi:10.11575/PRISM/30475.

[2] Yiannis N. Moschovakis. *Absolute lower bounds for uniform processes*. Draft notes, January 25, 2012. https://www.math.ucla.edu/~ynm/nadalb.pdf.

[3] Lou van den Dries and Yiannis N. Moschovakis. Is the Euclidean algorithm optimal among its peers? *Bulletin of Symbolic Logic* 10(3):390–418, 2004. doi:10.2178/bsl/1102022663.

[4] Lou van den Dries and Yiannis N. Moschovakis. Arithmetic complexity. *ACM Transactions on Computational Logic* 10(1), Article 2, 2009. doi:10.1145/1459010.1459012.

[5] Yiannis N. Moschovakis. *Abstract Recursion and Intrinsic Complexity*. Lecture Notes in Logic, vol. 48. Cambridge University Press and Association for Symbolic Logic, 2019. doi:10.1017/9781108234238.